

SureRide — Capstone Academic Reflection Essay

Author: Jeff Theuri

Institution: iLabAfrica, Strathmore University

Program: AI Practitioner Programme (AIPP)

Date: June 2026

Introduction: AI for Road Safety

Drunk driving remains a leading cause of tragic road fatalities in Kenya, with the National Transport and Safety Authority (NTSA) attributing over 30% of annual accidents to alcohol impairment. The primary barrier to safety is not a lack of concern, but rather the friction of booking a ride late at night when cognitively impaired. SureRide was conceived to close this specific gap by deploying an AI-powered conversational agent that makes ride booking as simple as sending a WhatsApp message. I expected the hardest part of this project to be the payment integration. What actually surprised me most was how consistently the ethical design decisions made early, for safety reasons turned out to produce cleaner engineering outcomes than the purely technical choices I agonised over. That inversion is the central thread of this reflection.

What Worked Well: Architecture & Grounding

The core technical execution rested on two pillars: **LangGraph** for conversation management and **ChromaDB + Google Gemini** for FAQ grounding. The stateful, deterministic dispatcher pattern in LangGraph proved highly successful as it kept the conversation on track, isolated user input from the LLM system prompt and made debugging trivial by ensuring every failure traced to a single handler function. RAG grounding ensured that policy queries such as pricing rules or service areas were answered strictly using verified company documents, eliminating hallucinated prices and policies entirely. In a manual audit of 50 sampled FAQ responses against the source document, I recorded zero factual errors, compared to an estimated 15–25% error rate typical of unconstrained LLM generation on the same query set. The WhatsApp-style UI, rendered inside a Streamlit iframe to isolate CSS, produced a familiar, low-cognitive-load interface suited for the impaired-user context. End-to-end booking from greeting to payment URL was consistently completed in under 60 seconds during testing.

Technical Challenges & Mitigations

Developing the application presented several genuine engineering hurdles:

1. **Streamlit Iframe Rendering:** Streamlit's layout engine strips custom CSS from `st.markdown()`, making a styled chat UI impossible through the standard route. I chose to render chat bubbles inside a `components.html()` iframe, which isolated the styles but introduced height estimation and scroll-injection complexity. These were resolved by dynamically calculating iframe height from message count and injecting a scroll script on each render cycle workable, but a reminder that framework constraints can silently redirect significant engineering effort.
2. **Payment Gateway Integration:** Transitioning from mock payment simulations to a live PesaPal v3 REST API required handling OAuth2 token rotation, IPN registration, and order submission in sequence. The deeper problem was that Streamlit's local server has no public callback URL, making PesaPal's webhook system entirely unusable in development. I mitigated this by building a manual polling button that queries the PesaPal status API directly using OAuth2 token validation a pragmatic solution, but one that leaves a real production gap requiring cloud deployment to close properly.
3. **LangGraph State Accumulation Bug:** This was the most instructive failure. The `add_messages` reducer in LangGraph appends messages rather than replacing them, which caused early versions of the dispatcher to re-trigger the `greet` node after every turn — the agent would re-introduce itself mid-booking. I spent several hours diagnosing this before identifying that the fix was a single guard condition: `not messages or messages[-1].type != "human"` at the top of the dispatcher. The lesson was not the fix itself but how long it took to find a sign that I had not fully read the LangGraph state management documentation before building on top of it.

Project Limitations & Honest Gaps

Despite its successes, the SureRide prototype has limitations that are honest constraints and not just scope decisions:

- **Geofencing:** Service is restricted to 19 Nairobi zones. This was a deliberate pilot decision, but it excludes peri-urban areas like Thika, Ruiru, Athi River where drunk-driving risk is just as acute.
- **Heuristic Name Parsing:** I extract the user's name by splitting on the first word and capitalising it. This fails reliably for compound names and greeting prefixes ("Hi, I'm John" → stored as "Hi"). A named-entity recognition model would fix this but was out of scope.
- **Language Barrier:** The agent responds only in English, which excludes a significant portion of Kenyan users. Gemini 1.5 Pro supports Swahili natively; this is the highest-priority localisation fix.
- **Unresolved Ethical Gap — Identity Verification:** The prototype accepts any name the user types with no OTP or account check. For a safety-critical service targeting impaired users, this is a meaningful vulnerability I identified but could not close within the 12-week

scope. The correct fix is a lightweight OTP tied to the M-Pesa number captured during PesaPal checkout which is architecturally straightforward but requires cloud deployment to implement properly. I consider this the single most important pre-production requirement.

Future iterations will address these in priority order: OTP verification, Swahili language support via a multilingual LLM, NER-based name extraction, Google Maps Directions API to replace the Haversine approximation, and KDPA-compliant PostgreSQL persistence for booking history.

Lessons Learned: Design Principles for Responsible AI

The most valuable lesson from this capstone was discovering that responsible AI design is not an afterthought but it must be embedded into every architectural decision from the start. The choice to use a hard-coded LangGraph dispatcher rather than a free-form LLM was driven as much by ethical reasoning (preventing hallucinated bookings) as by engineering preference. Delegating all payment processing to PesaPal's hosted pages rather than collecting card numbers directly was the correct KDPA-compliance call, even though it added integration complexity and cost me the webhook functionality entirely. In both cases, the ethical constraint produced a more robust architecture than the unconstrained alternative would have.

The LangGraph state bug and the identity verification gap are reminders that responsible design is not a one-time decision but a sustained discipline. I caught the state bug because I was testing systematically; I identified the identity gap because I was thinking about who would actually use this system at 2 AM. Both required stepping outside the code and asking a different kind of question. That habit of treating the user's context, not just the spec, as the primary design input is what the AIPP curriculum articulated in theory and what SureRide forced me to practise for real.
